

1 Introduction aux Side Effects

1.1 Qu'est-ce qu'un Side Effect ?

Information

Définition :

Un **side effect** (effet de bord) est une opération qui affecte quelque chose **en dehors** du composant React.

Exemples de side effects :

- Appels API (fetch, axios)
- Manipulation directe du DOM
- Timers (setTimeout, setInterval)
- Abonnements (websockets, événements)
- Logs (console.log)

1.2 Pourquoi les Side Effects sont un Problème ?

En React, le rendu doit être **pur** (sans effets de bord). Regardons le problème :

```
function BadComponent() {  
  // ERREUR : fetch dans le rendu !  
  fetch('https://api.example.com/data')  
    .then(res => res.json())  
    .then(data => console.log(data));  
  
  return <div>Composant</div>;  
}  
// Problème : fetch() sera appelé à CHAQUE rendu !  
// = Boucle infinie d'appels API
```

Listing 1 – Code INCORRECT - Side effect dans le rendu

Attention - Erreur Fréquente

Erreur fréquente :

Ne JAMAIS mettre de side effects directement dans le corps du composant. Ils seront exécutés à chaque rendu.

1.3 Solution : useEffect Hook

React fournit le hook `useEffect` pour gérer les side effects de manière contrôlée.

```
import { useEffect } from 'react';  
  
function GoodComponent() {  
  useEffect(() => {  
    // Side effect ici, exécuté APRÈS le rendu  
    fetch('https://api.example.com/data')  
      .then(res => res.json())  
      .then(data => console.log(data));  
  }, []); // [] = exécuter une seule fois  
  
  return <div>Composant</div>;  
}
```

Listing 2 – Code CORRECT - Side effect dans useEffect

2 useEffect Hook

2.1 Syntaxe de Base

IMPORTANT

Syntaxe useEffect :

```
import { useEffect } from 'react';

useEffect(() => {
  // Code du side effect

  return () => {
    // Cleanup (optionnel)
  };
}, [dependencies]);
```

Paramètres :

1. **Fonction callback** : Code du side effect
2. **Array de dépendances** : Quand exécuter l'effet

2.2 Array de Dépendances - Les 3 Cas

L'array de dépendances contrôle **QUAND** l'effet s'exécute.

2.2.1 Cas 1 : [] (Array Vide)

```
useEffect(() => {
  console.log('Exécuté une fois au montage');
}, []); // Array vide = une seule exécution
```

Listing 3 – Effet exécuté UNE SEULE FOIS

Quand l'effet s'exécute :

- Une seule fois, après le **premier rendu**
- Équivalent à `componentDidMount` (classe)

Cas d'usage typiques :

- Fetch initial de données
- Abonnement à un service
- Initialisation

2.2.2 Cas 2 : [dep1, dep2] (Avec Dépendances)

```
const [count, setCount] = useState(0);

useEffect(() => {
  console.log('Count a changé:', count);
}, [count]); // Exécuté quand count change
```

Listing 4 – Effet exécuté quand dépendances changent

Quand l'effet s'exécute :

- Au premier rendu
- À chaque fois qu'une dépendance change

Cas d'usage typiques :

- Fetch basé sur un filtre
- Synchroniser avec props/state
- Calculer valeurs dérivées

Attention - Erreur Fréquente

Règle d'or :

Toute variable utilisée dans useEffect DOIT être dans l'array de dépendances.

```
// ERREUR : userId utilisé mais pas dans dépendances
useEffect(() => {
  fetch('/users/${userId}')
}, []); // Oubli de [userId] !

// CORRECT
useEffect(() => {
  fetch('/users/${userId}')
}, [userId]); // userId dans dépendances
```

2.2.3 Cas 3 : Pas d'Array (Sans Dépendances)

```
useEffect(() => {
  console.log('Exécuté à CHAQUE rendu');
}); // Pas d'array = exécution à chaque rendu
```

Listing 5 – Effet exécuté à CHAQUE rendu

Quand l'effet s'exécute :

- Après CHAQUE rendu du composant

Attention - Erreur Fréquente

Attention :

Rarement utilisé ! Risque de boucles infinies. Préférez toujours un array de dépendances.

2.3 Résumé Array de Dépendances

Syntaxe	Exécution	Usage
[]	Une fois (montage)	Fetch initial
[dep]	Quand dep change	Fetch conditionnel
Pas d'array	Chaque rendu	Rarement utilisé

2.4 Cleanup Function

La fonction de cleanup (nettoyage) permet de libérer les ressources.

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log('Tick');
  }, 1000);

  // Cleanup : arrêter le timer
  return () => {
    clearInterval(timer);
  };
}, []);
```

Listing 6 – Cleanup function avec timer

Quand le cleanup s'exécute :

- Avant de ré-exécuter l'effet (si dépendances changent)
- Quand le composant se démonte

Cas d'usage cleanup :

- Arrêter timers (clearInterval, clearTimeout)
- Se désabonner (websockets, événements)
- Annuler requêtes en cours (AbortController)

Astuce**Bon à savoir :**

En développement avec Strict Mode, React exécute useEffect **deux fois** pour détecter les bugs. C'est normal et disparaît en production.